

CSCE 629 ANALYSIS OF ALGORITHMS · MODULE 03 · LECTURE NOTES

Randomization and Expectation

Buying worst-case robustness with coin flips.

MODULE OUTCOMES

- Explain why expected-case beats average-case as a guarantee.
- Use linearity of expectation, including on dependent variables.
- Analyse randomised quicksort and randomised selection.
- Distinguish Las Vegas from Monte Carlo algorithms.
- Apply concentration bounds to argue that deviation is unlikely.

1 The case for randomisation

The usual explanation — that randomised algorithms are faster on average — is not the real argument, and it understates what is going on.

Every deterministic algorithm has a worst case, and that worst case is a specific input. If the algorithm is public, the input is constructible. Deterministic quicksort taking the first element as pivot degrades to $\Theta(n^2)$ on already-sorted data, which is not an obscure adversarial construction — it is the most common shape real data takes.

WHAT RANDOMISATION ACTUALLY CHANGES

After randomising the pivot, the $\Theta(n^2)$ behaviour still exists. But it is no longer a property of the *input*; it is a property of an unlucky sequence of *your own coin flips*.

An adversary who knows your source code, your compiler, and your entire input history still cannot construct a slow case, because they cannot predict your randomness.

The guarantee changes from *fast unless the input is adversarial* to *fast with overwhelming probability, on every input*. That is a qualitative difference, and it is why randomisation is used in practice far beyond its speed benefits.

1.1 Las Vegas and Monte Carlo

	Las Vegas	Monte Carlo
Answer	Always correct	Correct with probability $\geq p$
Running time	A random variable	Bounded deterministically

	Las Vegas	Monte Carlo
Example	Randomised quicksort, quickselect	Miller–Rabin primality, Karger min-cut
Error handling	Wait longer	Repeat and take a majority / any success — error falls exponentially in the number of repeats

2 Linearity of expectation

$$E[X + Y] = E[X] + E[Y] \quad \text{for any } X, Y$$

No independence is required. This is worth stating emphatically, because it is the single fact that makes most expectation calculations in this course tractable, and almost everyone's instinct is that independence must be needed somewhere.

The standard technique: express the quantity you care about as a sum of **indicator** random variables, each 0 or 1. The expectation of an indicator is just the probability of its event, so:

$$X = \sum_i X_i \Rightarrow E[X] = \sum_i \Pr[\text{event } i \text{ occurs}]$$

You have replaced a hard question about a complicated random variable with many easy questions about single events, and you are allowed to do this even when the events are tangled together.

3 Randomised quicksort, analysed

Quicksort picks a pivot, partitions around it, and recurses on both sides. With a random pivot, how many comparisons does it make in expectation?

3.1 Setting it up

Label elements by their rank in sorted order: $z_1 < z_2 < \dots < z_n$. Define the indicator $X_{i,j} = 1$ if z_i and z_j are ever compared, 0 otherwise. Any two elements are compared at most once — a comparison only happens against a pivot, and a pivot is removed from all later subproblems. So the total comparison count is exactly $\sum X_{i,j}$, and by linearity:

$$E[\text{comparisons}] = \sum_{i < j} \Pr[z_i \text{ and } z_j \text{ are compared}]$$

3.2 The key observation

WHICH PIVOT IS CHOSEN FIRST FROM THE RANGE?

Consider $S = \{z_i, z_{i+1}, \dots, z_j\}$, the elements whose ranks lie between i and j inclusive. These stay together in the same subproblem until some element of S is chosen as a pivot.

Whichever element of S is chosen as a pivot **first** determines everything: if it is z_i or z_j , the two are compared against it and hence to each other. If it is anything strictly between them, they are separated into different subproblems and will never be compared.

Every element of S is equally likely to be the first of S chosen, and $|S| = j - i + 1$. Two of them are favourable. So:

$$\Pr[z_i \text{ and } z_j \text{ compared}] = 2 / (j - i + 1)$$

Summing gives $\sum_{i < j} 2/(j-i+1) \approx 2n \cdot H_n \approx 2n \ln n \approx 1.39 n \log_2 n$. The constant 1.39 is worth remembering: quicksort makes about 39% more comparisons than the information-theoretic minimum, and is still usually the fastest sort.

WHY QUICKSORT BEATS MERGESORT ANYWAY

Both are $\Theta(n \log n)$ in expectation, and quicksort has a worse worst case and makes more comparisons. It is still generally faster.

The reason is not in the comparison count. Quicksort partitions in place with two sequential scans, which is close to ideal for the prefetcher and uses $\Theta(\log n)$ extra space. Mergesort needs $\Theta(n)$ auxiliary space and writes to a different array than it reads.

This is Module 01's point made concrete: the model cannot see the difference, and the difference decides the benchmark.

4 Selection in linear time

Finding the k -th smallest element looks like it should require sorting, but it does not. **Quickselect** partitions exactly as quicksort does, then recurses into only the side containing rank k .

$$T(n) = T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n)$$

The work forms a geometric series $n + n/2 + n/4 + \dots < 2n$. Compare mergesort's $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$: the only difference is one recursive call instead of two, and it removes the logarithmic factor entirely. This is exactly the binary-search-versus-mergesort contrast from Module 02.

A deterministic linear-time algorithm also exists: **median of medians**, which chooses a provably decent pivot by recursively finding the median of groups of five. It is genuinely linear in the worst case and it is slower than quickselect on essentially every real input, because the constant factor is large. It is worth knowing as a proof that the worst case can be eliminated, and it is not worth using.

5 Concentration

An expectation on its own is a weak statement: a random variable with mean 10 could be 10 every time, or 0 half the time and 20 the other half. **Concentration bounds** say how unlikely it is to land far from the mean, and they are what make randomised algorithms trustworthy rather than merely good on average.

Bound	Requires	Gives	Decay
Markov	$X \geq 0$ only	$\Pr[X \geq a] \leq E[X]/a$	$1/a$ — very weak
Chebyshev	Known variance σ^2	$\Pr[X - \mu \geq k\sigma] \leq 1/k^2$	Polynomial
Chernoff / Hoeffding	Independent bounded variables	$\Pr[X \geq (1+\delta)\mu] \leq e^{-\delta^2\mu/3}$	Exponential

WHAT THIS MEANS IN PRACTICE

For randomised quicksort, a Chernoff-style argument shows the probability of exceeding $c \cdot n \log n$ comparisons is below $1/n^2$ for modest c .

At $n = 10^6$, that is under 10^{-12} . The machine is more likely to suffer an undetected memory error during the sort than the sort is to be slow.

At that point the worst case has stopped being an engineering concern. This is the difference between 'expected to be fast' and 'you may stop worrying', and it is why concentration bounds matter more than the expectation calculation they accompany.

6 Where randomisation recurs

Setting	What randomness buys	Where
Hashing	Defeats adversarial key choice; makes the uniformity assumption true by construction rather than by hope.	Module 05
Treaps, skip lists	Balance without rotation logic — far simpler code for the same expected bounds.	Module 05
Karger min-cut	A trivially simple algorithm that succeeds with probability $\geq 2/n^2$, repeated to amplify.	Module 11
Miller–Rabin	Practical primality testing, hence practical public-key cryptography.	CSCE 711
Sketching, streaming	Approximate answers in sublinear space.	CSCE 727
Monte Carlo rendering	Estimating the reflectance integral by sampling light paths. Variance reduction there is the same mathematics as here.	CSCE 647

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.046J — Randomized Algorithms, Quicksort analysis	The indicator-variable analysis of quicksort done carefully, plus Karger's min-cut.
Jeff Erickson, Algorithms — Randomized Algorithms chapter	Excellent on the Las Vegas / Monte Carlo distinction and on treaps.
Tim Roughgarden — Randomized Selection and QuickSort	Part 1. Unhurried derivation of the $2/(j-i+1)$ probability.
Mitzenmacher & Upfal, lecture slides (free)	Concentration bounds with worked applications, if you want the probability theory tightened.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement deterministic quicksort (first element as pivot) and randomised quicksort. Run both on sorted, reverse-sorted, and random input of size 10^5 . Report the times and explain the pattern.
- 2 Verify the comparison count empirically: instrument randomised quicksort to count comparisons, average over 100 runs at several n , and check the result against $1.39 n \log_2 n$.
- 3 Implement quickselect and verify experimentally that it is linear by fitting a log-log slope.

- 4 Implement median-of-medians selection. Confirm it is linear, then measure it against quickselect and report the constant-factor ratio.
- 5 Use linearity of expectation to compute the expected number of fixed points in a random permutation of n elements. The answer is 1 regardless of n ; explain why, and verify by simulation.
- 6 Simulate 10,000 runs of randomised quicksort at $n = 10^4$ and plot the distribution of comparison counts. Observe how tightly it concentrates, and relate this to §5.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does randomisation actually remove, given that the worst case still exists?
- 2 State the difference between average-case and expected-case analysis and say which is stronger, with a reason.
- 3 Does linearity of expectation require independence? Give an example from this module where the summands are clearly dependent.
- 4 Derive $\Pr[z_i \text{ and } z_j \text{ are compared}] = 2/(j-i+1)$.
- 5 Why is quickselect linear while quicksort is $n \log n$, given that they partition identically?
- 6 What does a Chernoff bound add to an expectation result, and why does exponential rather than polynomial decay matter?